

Yes — **there is a genuinely interesting idea here**, but I would change one fundamental part of your framing:

Don't try to build an AI that "predicts the next stock price."
Build an AI that estimates **the probability distribution of future outcomes under defined market scenarios**.

That is much more realistic, technically interesting, and potentially useful.

Your intuition is actually close to how sophisticated quantitative investing works: instead of asking "*what will happen?*", ask "*given the information available today, what outcomes have historically followed, under what conditions, and with what probability?*"

My assessment

Idea: 8/10

Potential research value: 9/10

Difficulty: 9/10

Chance of making a model that consistently predicts exact prices: very low

Chance of making a useful scenario/risk-ranking system: considerably better

1. Your core idea is valid

Suppose we're analysing an Indian steel company.

Instead of giving the AI just:

Stock price + financial statements → predict next price

you give it a **state of the world**:

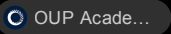
Parameter	Current state
Global steel price	+12%
Domestic steel price	+8%
China exports	High
India infrastructure spending	Strong
Coal/coking coal cost	+5%

Parameter	Current state
INR/USD	84
Export demand	Rising
Domestic demand	Strong
Interest rates	Stable
Company capacity utilisation	87%
Company margins	Improving
Inventory	Low
Debt	Moderate
Valuation	18× PE
Sector momentum	Positive

Now ask:

"Find historical periods where the market state looked similar and determine what happened subsequently."

That's a much more defensible problem.

And ML is actually capable of finding nonlinear interactions between variables. A major empirical asset-pricing study found that ML models can exploit nonlinear interactions among hundreds of stock characteristics and macro variables for out-of-sample return forecasting. 

2. But I wouldn't use a Vector DB as the "brain"

This is the first major architectural change I'd make.

A vector database is excellent for **retrieval**, but it isn't itself the predictive model.

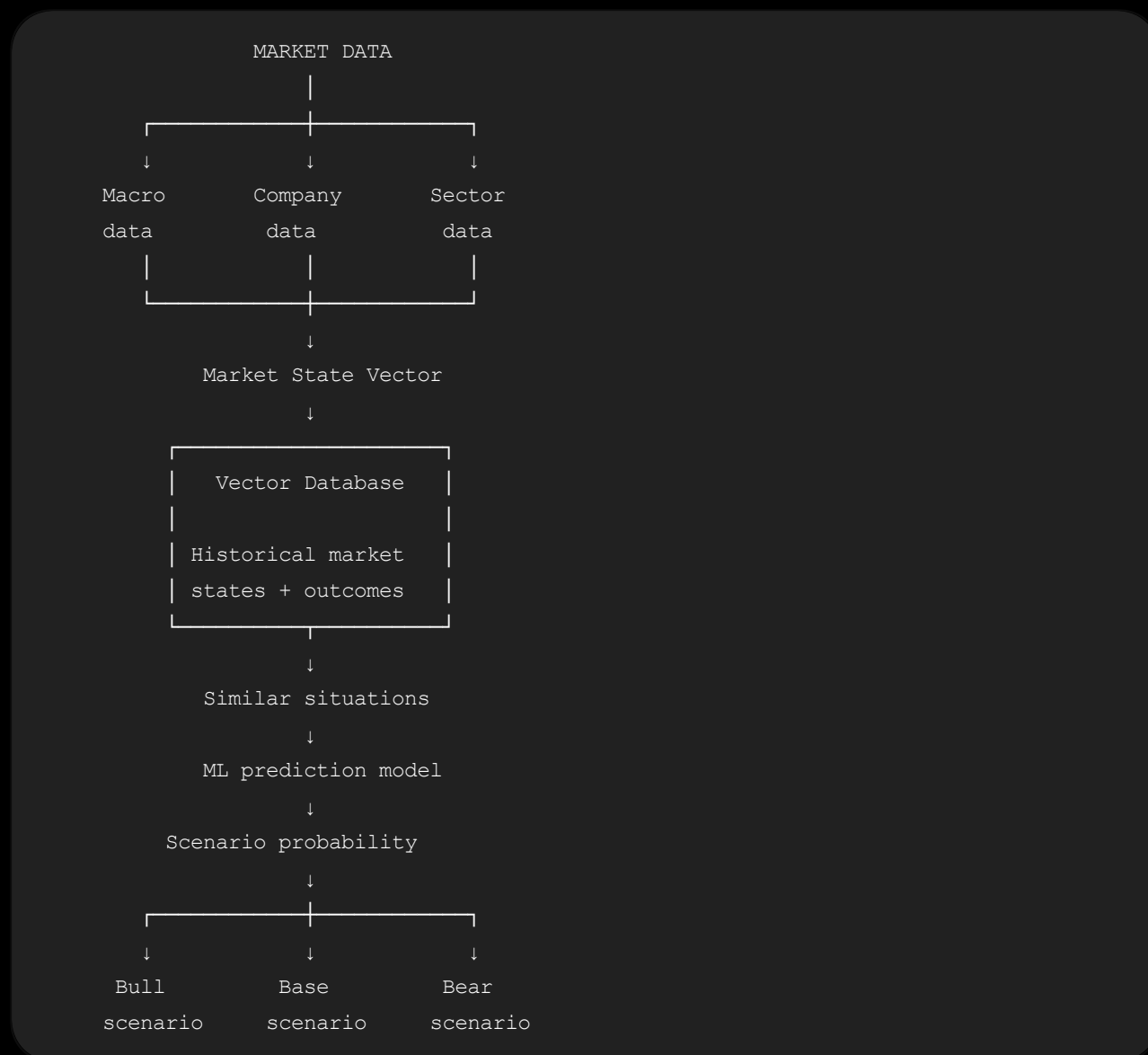
Think of your system as:

Data → State representation → Retrieval → Prediction → Scenario engine

rather than:

Data → Vector DB → prediction

For example:



This is where your idea becomes interesting.

3. The really interesting part: "market state"

I think **this should be the heart of your project**.

Instead of storing historical data simply as:

Date → Stock price

you create a **market state snapshot**.

For example:

DATE: 2019-08-01

```
Global demand:      -0.3
Domestic demand:    +0.6
Commodity prices:    -0.2
USD/INR:             +0.4
Interest rates:      -0.5
Inflation:           +0.2
Exports:             -0.4
Sector momentum:     +0.7
Company earnings:    +0.5
Valuation:           -0.1
Liquidity:           +0.3
Market sentiment:    -0.6
...
```

Then associate it with future outcomes:

```
T0 state
  |
  ├── +1 month return
  ├── +3 month return
  ├── +6 month return
  ├── +12 month return
  ├── maximum drawdown
  ├── volatility
  └── sector-relative return
```

Now you have a dataset of:

Market state → future distribution

That's considerably more powerful than simply training on prices.

4. Your "astrology" analogy is interesting — but I'd change it

Your analogy:

"We try to predict human behaviour by studying star positions at birth."

is useful as a conceptual analogy, but **don't use astrology as the scientific foundation.**

There's an important difference.

Astrology assumes a relationship between variables that doesn't have a demonstrated causal mechanism.

Your model would instead say:

"When these observable economic/financial conditions occurred historically, these types of outcomes tended to follow."

That's empirically testable.

So your model isn't really:

stars → human behaviour

It is:

economic state → probability distribution of market outcomes

And you can continuously test whether that relationship survives out-of-sample.

5. I would actually use 3 different models

This is where I'd make your project substantially better.

Model A — State similarity engine

This is where the vector database comes in.

Given today's conditions:

```
India demand      = strong
Global demand     = weak
Commodity cost    = rising
USDINR            = weakening
Interest rate     = stable
Company earnings  = accelerating
Valuation         = expensive
...
```

find historical states that resemble today.

For example:

```
Historical analogue #1
Similarity: 93%
2007 Q3

Historical analogue #2
Similarity: 89%
2016 Q2
```

Historical analogue #3
Similarity: 87%
2021 Q1

Then examine what happened afterward.

6. Model B — Predictive model

Don't make the vector search itself predict returns.

Feed the structured variables into ML models.

I'd start surprisingly simple:

Baselines

- Linear regression
- Logistic regression
- Random Forest
- XGBoost/LightGBM
- Neural network

Then compare them.

You might discover something important:

Your fancy neural network isn't actually better than XGBoost.

That's extremely common in structured financial data.

In fact, research has found that relatively simple regression approaches can sometimes perform comparably to much more complex ML approaches, particularly after considering costs. 

7. Model C — Scenario engine

This is the part I think could make your project special.

Instead of:

"Stock XYZ will go to ₹1,420."

your system says:

Current probability distribution

Next 6 months

Scenario	Probability	Expected return
Strong bull	18%	+31%
Bull	27%	+17%
Base	35%	+6%
Bear	15%	-14%
Severe bear	5%	-29%

Then explain:

The strongest positive drivers are domestic demand, improving margins and declining input costs.

And:

The biggest downside risks are commodity inflation, currency depreciation and expensive valuation.

That is far more useful than pretending to know the exact future price.

8. Even better: let the user change the world

Imagine your interface has sliders.

What if?

Global demand		+10%
Domestic demand		+15%
Commodity prices		+5%
INR depreciation		+2%
Interest rates		unchanged
Export growth		+12%

Then:

Run scenario

And your model produces:

```
Sector impact:      +11.4%
Company earnings:   +15.8%
Expected return:    +13.2%
```

```
Probability of positive return: 71%
```

```
Primary drivers:
```

1. Domestic demand
2. Export growth
3. Margin expansion

```
Primary risks:
```

1. Commodity prices
2. Currency

Now you have something closer to a **financial simulation engine** than a generic "AI stock predictor."

9. Your 10–15 parameters need to be carefully designed

I wouldn't simply choose 15 variables because 15 sounds manageable.

I'd create **hierarchical factors**.

Level 1 — Global

- Global GDP/demand
- Commodity prices
- US rates
- Dollar
- China
- Global liquidity

Level 2 — India

- GDP
- CPI/WPI
- RBI rates
- INR
- fiscal spending
- credit growth

- domestic consumption

Level 3 — Sector

For steel:

- steel prices
- coking coal
- iron ore
- imports
- exports
- capacity utilisation
- infrastructure demand

For IT:

- USD/INR
- US IT spending
- hiring
- deal wins
- wage inflation
- utilisation
- attrition

Level 4 — Company

- revenue growth
- EBITDA margin
- debt
- ROCE
- cash flow
- valuation
- promoter holding
- institutional ownership
- earnings revisions

This gives you something like:

GLOBAL



INDIA



SECTOR



COMPANY



STOCK

That hierarchy is extremely important.

10. Don't only train on stock prices

This is probably the biggest mistake I'd warn you about.

You need **causal/event information as well as numerical data**.

For example:

```
2020:
COVID
  ↓
lockdowns
  ↓
demand collapse
  ↓
commodity prices collapse
  ↓
earnings decline
  ↓
market crash
  ↓
massive monetary stimulus
  ↓
recovery
```

Your system should know that this was an unusual regime.

Similarly:

```
Russia-Ukraine war
→ energy shock
→ commodity inflation
→ sector-specific winners/losers
```

or:

```
China slowdown
→ commodity demand
→ metals
→ mining
→ shipping
→ Indian exporters
```

This is where an LLM + vector database can actually add value.

11. This is where your vector database becomes extremely useful

Store historical **events and situations**, not just numerical vectors.

For example:

Document:

2018 Emerging Market Selloff

Macro:

- USD strengthening
- US rates rising
- Emerging market currencies weakening
- Oil rising

India:

- INR depreciation
- FII outflows
- inflation concerns

Sector:

- Financials under pressure
- IT exporters benefiting

Outcome:

NIFTY: -X%

Banking: -Y%

IT: +Z%

...

Embed that.

Then today's state can retrieve historically similar situations.

So your vector DB becomes a:

Historical market memory

rather than a magical prediction engine.

I really like that architecture.

12. You could create a "Market DNA"

This could actually become the conceptual identity of your system.

Every day, every stock gets a **Market DNA vector**.

For example:

TATA STEEL

```
[0.72, -0.31, 0.81, 0.42, -0.17,  
0.66, 0.35, -0.22, 0.73, ...]
```

This vector represents:

"What does the environment surrounding this company look like right now?"

Then you compare today's DNA against millions of historical states.

That gives you:

Today → historically similar situations → outcomes

That's a very compelling research project.

13. But there's a huge trap: data leakage

This is probably the **#1 technical danger** in your project.

Suppose you're training using information from:

January 2018.

You must only use information that was actually available **on January 31, 2018**.

You cannot accidentally use:

- revised financial statements
- later earnings information
- future macro revisions
- survivorship-biased stock lists
- future corporate actions
- future index membership

Otherwise your model will look incredible in backtesting and completely fail in reality.

This is one reason stock ML research is notoriously difficult to evaluate properly. Recent literature continues to highlight problems around evaluation methodology and transaction

14. Don't measure "accuracy"

This is another important change.

Don't ask:

"Was my prediction correct 70% of the time?"

Instead measure:

Directional accuracy

Did the model correctly predict up/down?

Expected return

How close was expected return to actual return?

Calibration

If model says:

70% probability of positive return

does that actually happen roughly 70% of the time?

Sharpe ratio

Does a strategy based on the predictions produce risk-adjusted returns?

Maximum drawdown

How badly can it lose?

Turnover

How frequently does it trade?

Transaction costs

Does the strategy still work after costs?

This last part is critical. Some ML strategies that look excellent before costs become much less attractive after trading costs. SSN SS...

15. I'd also avoid predicting absolute prices initially

Start with:

Target 1

$P(\text{Return} > 0 \mid \text{current state})$

Then:

Target 2

Expected 3-month return

Then:

Target 3

Probability of >10% return

Then:

Target 4

Probability of >10% drawdown

Eventually:

$P(\text{Return} < -20\%)$

$P(-20\% < \text{Return} < 0\%)$

$P(0\% < \text{Return} < 10\%)$

$P(10\% < \text{Return} < 25\%)$

$P(\text{Return} > 25\%)$

This produces a **probability distribution**, not a point prediction.

That's much more appropriate for financial markets.

16. One more improvement: predict relative performance

Instead of asking:

"Will Reliance rise?"

ask:

"Will Reliance outperform NIFTY over the next 6 months?"

That's often a cleaner problem.

For example:

Expected NIFTY return:	+7%
Expected Reliance return:	+14%
Expected alpha:	+7%

Or:

IT sector:	+12%
Company:	+18%
Expected sector-relative return:	+6%

This removes some of the noise from predicting the entire market.

17. Your first MVP should be much smaller

I wouldn't start with:

"AI that predicts the Indian stock market."

That's enormous.

Start with something like:

"Historical scenario engine for Indian metals stocks."

Maybe:

10 stocks

10 years

15–25 variables

daily/weekly observations

3 prediction horizons

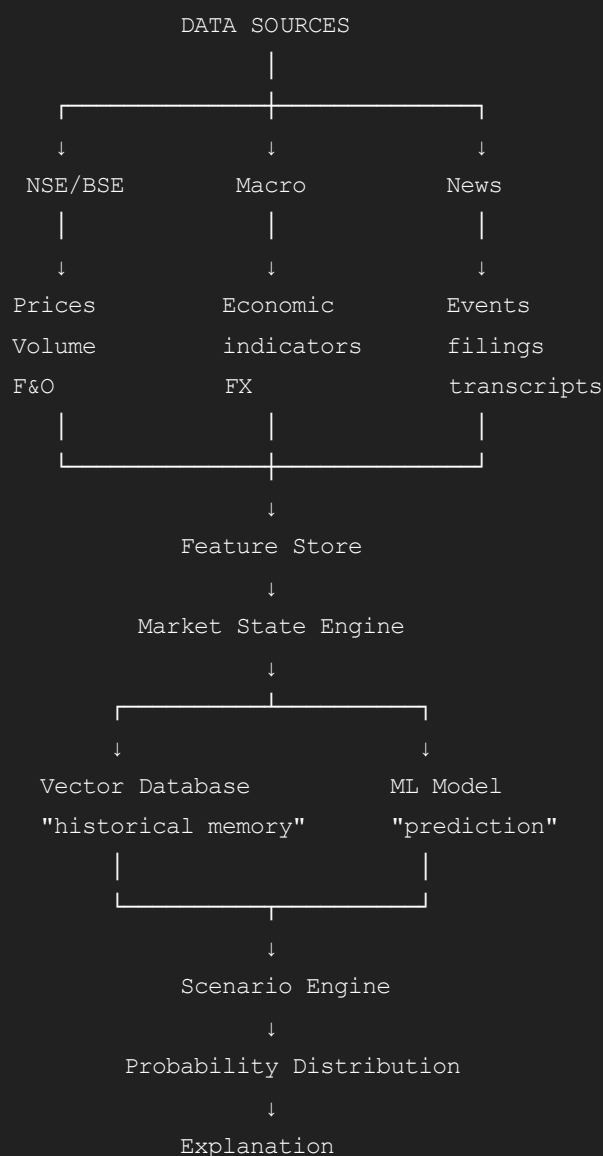
- 1 month
- 3 months
- 6 months

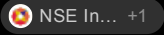
And compare:

```
Vector similarity
  vs
XGBoost
  vs
Neural network
  vs
Simple baseline
```

Then you'll actually learn something.

18. Your data architecture could look like this



NSE itself provides historical EOD, trade/order, price-volume and related market datasets, so there is a legitimate foundation for building the historical layer, although licensing/access requirements need to be considered. 

19. The most important conceptual distinction

I'd divide your system into three questions:

What is happening?

LLM + data pipeline



State estimation

What has happened in similar situations?

Vector DB



Historical analogues

What is likely to happen?

ML/statistical model



Probability distribution

Then:

What if something changes?

Scenario engine



Counterfactual analysis

That final component is where your idea becomes particularly interesting.

20. And there is a deeper research question here

You could eventually test:

Does conditioning historical market returns on a sufficiently rich representation of market state provide better out-of-sample predictions than conventional factor models?

That's a legitimate quantitative-finance research question.

You could even compare:

Model 1

Price/technical indicators only

vs.

Model 2

Fundamentals

vs.

Model 3

Macro + fundamentals

vs.

Model 4

Macro + fundamentals + events

vs.

Model 5

Macro + fundamentals + events + vector historical retrieval

vs.

Model 6

Everything + scenario engine

Then measure whether each additional layer actually improves out-of-sample performance.

If the vector database doesn't improve prediction, **that's also a valuable result.**

My biggest recommendation

Don't call it an "AI stock predictor."

I'd think of it as:

Market State & Scenario Intelligence Engine

The fundamental proposition becomes:

"Given today's economic, sectoral and company conditions, find historically comparable market states, learn how markets behaved under those conditions, and estimate the probability distribution of future outcomes under alternative scenarios."

That is a **much stronger idea** than "use AI to predict stocks."

And importantly, it doesn't require believing that markets are perfectly predictable. You're trying to extract **conditional information from noisy distributions**, which is much closer to what the academic literature actually supports. [OUP Acade...](#)

One final practical point: if you eventually turn this into a product that gives investment recommendations or automated trading signals in India, you'll need to consider SEBI's rules around AI/ML investor-facing applications and algorithmic trading; SEBI has specifically addressed AI/ML systems used by market intermediaries and has an evolving framework around retail algo trading. [Securities and ... +1](#)

If you want to actually build this, I think the next step should be designing the system scientifically before writing code: define the 15–25 variables, exact historical data schema, target variables, vector representation, ML models, backtesting methodology, and MVP architecture. That's where we can turn your idea into a concrete engineering/research project.